Technical Report

pyedger: a Python implementation of edgeR that reproduces Bioconductor results to numerical precision

Zehua Zeng,[1,2,*] **Meng Xu,**[3] **and Hongwu Du**[4,5,*]
[1]Department of Genetics, Stanford University, Stanford, CA 94305, USA.
[2]OmicVerse, Inc., San Francisco, CA, USA.
[3]Hubei Key Laboratory of Agricultural Bioinformatics, College of Informatics, Huazhong Agricultural University, Wuhan 430070, China.
[4]School of Chemistry and Biological Engineering, University of Science and Technology Beijing, Beijing 100083, China.
[5]Daxing Research Institute, University of Science and Technology Beijing, Beijing 100083, China.
*Correspondence: starlitnightly@gmail.com, hongwudu@ustb.edu.cn

Summary

We present pyedger, a pure-Python implementation of the edgeR negative-binomial workflow for differential expression of count data, covering TMM normalisation, filterByExpr, estimateDisp, the quasi-likelihood (QL) F-test, the likelihood-ratio test and the classic exact test. Rather than aiming for similar results, pyedger ports edgeR's compiled code paths and its limma and locfit dependencies step by step and is versioned against a specific edgeR release. On eight TCGA tumour-normal comparisons (4,972 samples) and 60 small-sample analyses it returns the same differentially expressed genes as edgeR 4.10.5 in all three tests, P-values within 1.2e-5 relative difference and identical gene-set enrichment results, while running 1.1-1.3 times faster than edgeR on one thread and 3.4-3.5 times faster on eight. The benchmark also exposed a tail-probability underflow in the exact test and several API differences from R in pyedger 0.2.2, which are fixed in 0.2.3.

Keywords edgeR, differential expression, RNA-seq, negative binomial, Python, reproducibility

Introduction

edgeR[1,2] is one of the most widely used methods for differential expression (DE) analysis of RNA-seq and other count data. It is written in R and C, and Python users have so far reached it through `rpy2`, which requires a parallel R installation, converts data back and forth and hides edgeR's options behind a binding layer[3]. PyDESeq2[3] showed that a native re-implementation of DESeq2 is valuable, but also that it "yields similar, but not identical, results". For a method that users routinely run in both languages, and whose results end up in publications, the bar should be higher: a user who switches from R to Python should get the same genes, the same P-values and the same pathways.

pyedger was written to meet that bar. This report documents what it implements, how its numerical agreement with edgeR was achieved, how it behaves on real data of very different sizes, what the benchmark found in the previous release, and what a user moving from R should know.

Implementation

Features and code structure

pyedger 0.2.3 implements edgeR 4.10's count-based DE workflow (Table 1): the `DGEList` container, TMM, TMMwsp, RLE and upper-quartile normalisation, `filterByExpr`, `cpm` and `aveLogCPM`, common, trended and tagwise dispersion estimation with and without a design matrix (Cox-Reid adjusted profile likelihood, weighted-likelihood empirical Bayes), robust dispersions, GLM fitting with the likelihood-ratio test, `glmTreat`, the classic exact test, and the QL pipeline in both its legacy form and the bias-adjusted form introduced in edgeR 4[2]. Functions are exported under Python names (`estimate_disp`) and their R names (`estimateDisp`), and results are pandas data frames with edgeR's column names.

The package depends on NumPy, SciPy, pandas and numba, whose compiled kernels run the Levenberg GLM fits and the QL adjustments. Genes are processed in blocks on a thread pool; runs with one and eight threads give bit-identical results.

How numerical parity is obtained

Most of edgeR's numerical work happens in compiled code and in two other packages, limma (hyper-parameter estimation) and locfit (the dispersion trend). pyedger follows each of them step by step rather than re-deriving the statistics: edgeR's Levenberg-damped GLM fit and its one-way shortcut, the Cox-Reid adjusted profile likelihood with edgeR's LDL floor, the zero-fit handling with reduced designs, `q2qnbinom` with log-scale gamma tail probabilities, the adaptive locfit tree and vertex interpolation, limma's `squeezeVar` family (`fitFDist`, `fitFDistUnequalDF1`, `fitFDistRobustly`), R's Brent `optimize` and `uniroot`, and the

© 2026 The Authors
Published at omicverse.org/articles/pyedger, version 1.

Table 1. edgeR functions and their pyedger counterparts (R names are exported as aliases).

edgeR	pyedger	Largest difference from edgeR 4.10.5 on TCGA
normLibSizes	calc_norm_factors	norm. factors 4.9×10^{-15}
filterByExpr	filter_by_expr	same genes kept
aveLogCPM	ave_log_cpm	5×10^{-12}
estimateDisp	estimate_disp	tagwise dispersion 8.6×10^{-10}
glmFit, glmLRT	glm_fit, glm_lrt	P-value 3.8×10^{-7}
glmQLFit, glmQLFTest	glm_ql_fit, glm_qlf_test	P-value 1.2×10^{-5}
exactTest	exact_test	P-value 5.1×10^{-8}

edgeR 4 QL adjustments. Details that change results on real data were found by comparing against R on real data, not only on simulations; for example, TMM takes $\log_2$ through the C library because NumPy's SIMD $\log_2$ differs by one ulp and TMM's trimming on heavily tied log-ratios is sensitive to that ulp. The repository's 42 tests check each step against reference values regenerated from edgeR 4.10.5; regenerating the references for this report reproduced all pre-existing reference files byte for byte.

Comparison with edgeR on TCGA

Data and protocol

We used STAR gene counts of eight TCGA projects (BRCA, COAD, KIRC, LIHC, LUAD, LUSC, PRAD and THCA; UCSC Xena GDC hub) and compared primary tumours with solid-tissue normals, one aliquot per participant (421 to 1,208 samples per cohort, 4,972 in total). Xena stores $\log_2(\text{count} + 1)$, from which the integer counts were recovered; samples whose values were averages of replicate aliquots, and therefore not integer counts, were removed. To cover the small-sample regime in which edgeR is mostly used, we also drew 20 random subsets each of 3, 5 and 10 normals versus as many tumours from TCGA-LUAD. Both packages ran the same default edgeR pipeline: `filterByExpr`, TMM, `estimateDisp` with design ~group, then `glmQLFit` and `glmQLFTest`, `glmFit` and `glmLRT`, and `exactTest` (22,730 to 27,801 genes retained per cohort). Genes were called DE at FDR ≤ 0.05 and $|\log_2 \text{FC}| \geq 1$.

The same genes

The QL F-test called 61,578 DE genes in total across the eight cohorts with edgeR, and pyedger's calls were identical in every cohort (Fig. 1a); so were those of the likelihood-ratio test and the exact test. No gene crossed the FDR 0.05 threshold in one package but not the other in any test, and the $-\log_{10} P$ values lie on the identity line in every cohort (Fig. S1).

The same pathways

With the same fgsea seed, the two packages gave identical enrichment calls for all 400 cohort-pathway pairs (257 significant), with NES differing by at most 2.5×10^{-8} (Fig. 1b, blue). As a reference for what "different" means here, re-running fgsea on edgeR's own statistics with another seed changed 8 calls and moved NES by up to 0.08 (Fig. 1b, grey): the difference between pyedger and edgeR is far below the Monte-Carlo noise of the downstream analysis.

Numerical agreement

Normalisation factors agree to 4.9×10^{-15}, common dispersions to 1.1×10^{-12} and the empirical-Bayes prior degrees of freedom to 5.6×10^{-10}. Per gene, tagwise dispersions agree to 8.6×10^{-10}, exact-test P-values to 5.1×10^{-8} and likelihood-ratio P-values to 3.8×10^{-7} (Fig. 1c); log-fold changes differ by at most 3.3×10^{-8} in absolute terms (relative differences look larger only for log-fold changes close to zero). QL P-values have a median relative difference of 3.7×10^{-9} and a maximum of 1.2×10^{-5}; the larger tail comes from the QL dispersion, which edgeR estimates with `optimize(tol=1e-5)` on a transformed scale, so it is only defined to about four significant digits in R itself. In the 60 small-sample analyses the largest relative P-value difference was 1.3×10^{-6} and no DE call differed (Fig. 1e).

Speed

On one thread pyedger took 0.76 to 0.89 of edgeR's time, and with eight threads it ran the full pipeline 3.4 to 3.5 times faster than edgeR, which is single-threaded (BRCA, 1,208 samples: 277 s and 102 s against 351 s; Fig. 1d). Most of the gain comes from not repeating work that edgeR repeats: the 21-point dispersion grid of `estimateDisp` refits the same counts at 21 dispersions, and pyedger builds the group sub-matrices, e^{offset}, $\log \Gamma(y+1)$ and the per-gene dispersion terms of the adjusted profile likelihood once, uses $e^{\beta} e^{\text{offset}}$ instead of one exponential per observation in each Newton step, and skips the deviance, which the grid never uses. Timings are single runs on a node shared with other jobs, so differences of 10-20% should not be over-interpreted.

What the benchmark found in pyedger 0.2.2

The repository's parity tests, built mainly on a simulated 600-gene data set, all passed for pyedger 0.2.2. Running this benchmark on 0.2.2 found problems that only real data of this size reach:

1. **Exact test.** Six genes in TCGA-BRCA with $\log_2$FC between −4.5 and −7 received $P = 1$ instead of edgeR's $P \approx 0$. edgeR maps counts to pseudo-counts with `q2qnbinom`, which calls `pgamma` and `qgamma` with `log.p = TRUE`; pyedger took the logarithm of SciPy's `gammainc`, which underflows to zero in the far tail, so the pseudo-count became infinite. pyedger 0.2.3 evaluates both functions in log space (series and continued-fraction expansions where SciPy underflows, Newton steps on the log scale for the inverse) and matches `q2qnbinom` to 10^{-8} on 2,000 extreme inputs. The same tails produced the `RuntimeWarning`s users saw from `log(gammainc)`; the test suite had silenced `RuntimeWarning`, which is why the problem went unnoticed. Separately, edgeR keeps negative pseudo-counts, so a group sum can exceed the total and R's `s1:s` then counts downwards; pyedger summed over an empty range and returned $P = 0$ for five TCGA-COAD genes whose edgeR P-values are

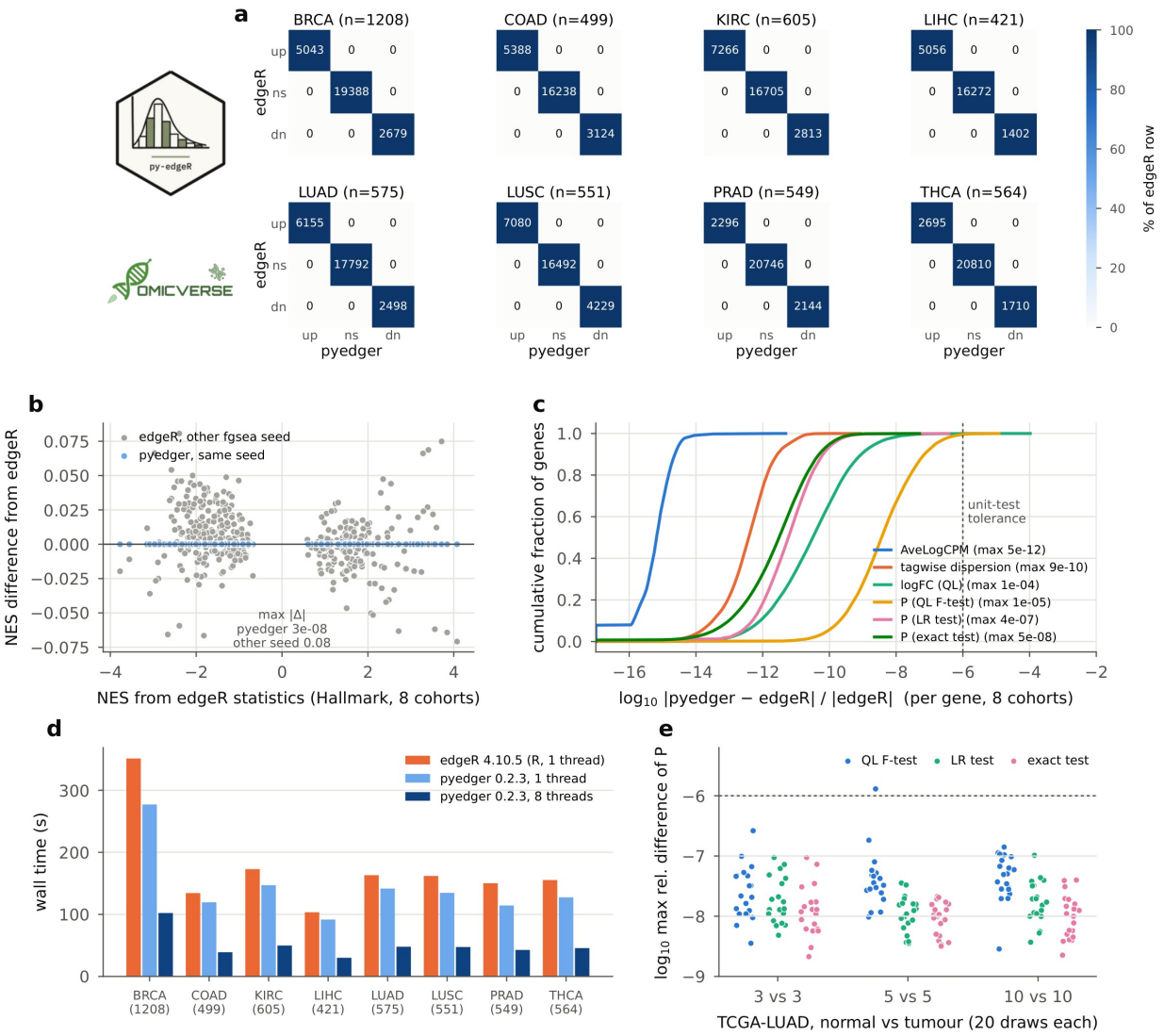


Figure 1. pyedger 0.2.3 versus edgeR 4.10.5 on TCGA tumour-normal comparisons.

(a) DE calls of the QL F-test (FDR ≤ 0.05, $|\log_2 \mathrm{FC}| \geq 1$); numbers are genes, colour the percentage of the edgeR row. **(b)** Gene-set enrichment (fgsea[4], MSigDB Hallmark, 50 sets $\times$ 8 cohorts), genes ranked by $\mathrm{sign}(\log \mathrm{FC})\sqrt{F}$. Each point is one cohort-pathway pair: its NES from edgeR's statistics (x) and the difference to it (y) of the NES from pyedger's statistics with the same seed (blue), or from edgeR's statistics with another seed (grey), which shows fgsea's own Monte-Carlo variation. **(c)** Cumulative distribution of per-gene relative differences between pyedger and edgeR, pooled over the eight cohorts; values below 10^{-17} are drawn at 10^{-17}. **(d)** Wall time of the full pipeline (normalisation to all three tests), one run each; the single-threaded runs were made concurrently, one process per cohort, on the same node, and pyedger with eight threads ran alone. **(e)** Largest relative P-value difference per random draw in the small-sample regime (TCGA-LUAD).

10^{-16} to 10^{-28}.

2. **Speed.** pyedger 0.2.2 took 1.16 to 1.25 times edgeR's time on one thread, and eight threads did not help (0.79 to 1.26 times; BRCA 443 s), because with a two-group design the GLMs go through the one-way shortcut, which in 0.2.2 ran serially and recomputed the same quantities at every grid point.

3. **API.** The README example failed: `DGEList` could not be subset, and the second argument of `estimateDisp` was `group` rather than the design. `exactTest` rejected a numeric dispersion vector and `TMMwsp` normalisation raised an error. All four now behave as in edgeR.

Every fix comes with a regression test against edgeR 4.10.5, including the TCGA-BRCA and TCGA-COAD genes above, which fail on 0.2.2.

Notes for users coming from R

1. **Versions.** pyedger tracks a specific edgeR release (0.2.x: edgeR 4.10.5). Since edgeR 4.2 changed the default QL method, results from older edgeR versions differ from both edgeR 4.10 and pyedger; `glmQLFit(legacy=True)` reproduces the classic method. Use pyedger 0.2.3 or later.

2. **Indices are 0-based.** `glmQLFTest(fit, coef=1)` in Python is `coef = 2` in R; coefficient names are also accepted.

3. **Subsetting.** `y[keep]` keeps the original library sizes, as R's default does; the usual `y[keep, , keep.lib.sizes = FALSE]` after `filterByExpr` is `y.subset(keep, keep_lib_sizes=False)`.

4. **Results agree to the optimiser, not to the last bit.** edgeR estimates several quantities with `optimize` or `uniroot` at fixed tolerances, so P-values agree to about 10^{-5} relative at worst, far below anything that changes a DE call.

5. **Scope.** Count-based DE in edgeR is covered. edgeR's plotting functions, `voom`-style limma workflows, `diffSpliceDGE`, `goana`, `camera`, `roast` and the bisulfite methylation helpers are not part of pyedger.

Conclusion

pyedger gives Python users edgeR's results, not an approximation of them: on eight real cohorts of 421 to 1,208 samples and on 60 small-sample draws it reproduces edgeR's DE genes and pathways, and its statistics to the precision of edgeR's own optimisers, while running faster than edgeR on one thread and several times faster with eight. Validating on real data at scale, rather than on the small simulations of the unit tests, was what found the remaining defects. pyedger is the edgeR backend of OmicVerse (`ov.bulk.pyDEG` and Milo), which installs it from PyPI, so OmicVerse users run the package validated here.

Data and code availability

pyedger is released under LGPL-3.0-or-later on PyPI (`pip install pyedger`) and at https://github.com/omicverse/py-edgeR. The benchmark (`report/scripts`) rebuilds every number and figure in this report. TCGA STAR counts were obtained from the UCSC Xena GDC hub (https://gdc-hub.s3.us-east-1.amazonaws.com) and MSigDB Hallmark 2020 gene sets from Enrichr, both accessed on 9 November 2026. Software: pyedger 0.2.3 (Python 3.10, NumPy 2.2.6, SciPy 1.15.2, numba 0.67.0); edgeR 4.10.5, limma 3.68.5, locfit 1.5-9.12 and fgsea 1.36.2 on R 4.5.3 (bioconda). Hardware: one node of the Stanford Sherlock cluster (Intel Xeon Platinum 8462Y+, 20 cores allocated), shared with other jobs.

References

[1] Robinson MD, McCarthy DJ, Smyth GK (2010). edgeR: a Bioconductor package for differential expression analysis of digital gene expression data. Bioinformatics 26, 139-140. 10.1093/bioinformatics/btp616

[2] Chen Y, Chen L, Lun ATL, Baldoni PL, Smyth GK (2025). edgeR v4: powerful differential analysis of sequencing data with expanded functionality and improved support for small counts and larger datasets. Nucleic Acids Research 53, gkaf018. 10.1093/nar/gkaf018

[3] Muzellec B, Teleńczuk M, Cabeli V, Andreux M (2023). PyDESeq2: a python package for bulk RNA-seq differential expression analysis. Bioinformatics 39, btad547. 10.1093/bioinformatics/btad547

[4] Korotkevich G, Sukhov V, Budin N, Shpak B, Artyomov MN, Sergushichev A (2021). Fast gene set enrichment analysis. bioRxiv. 10.1101/060012

Supplementary information

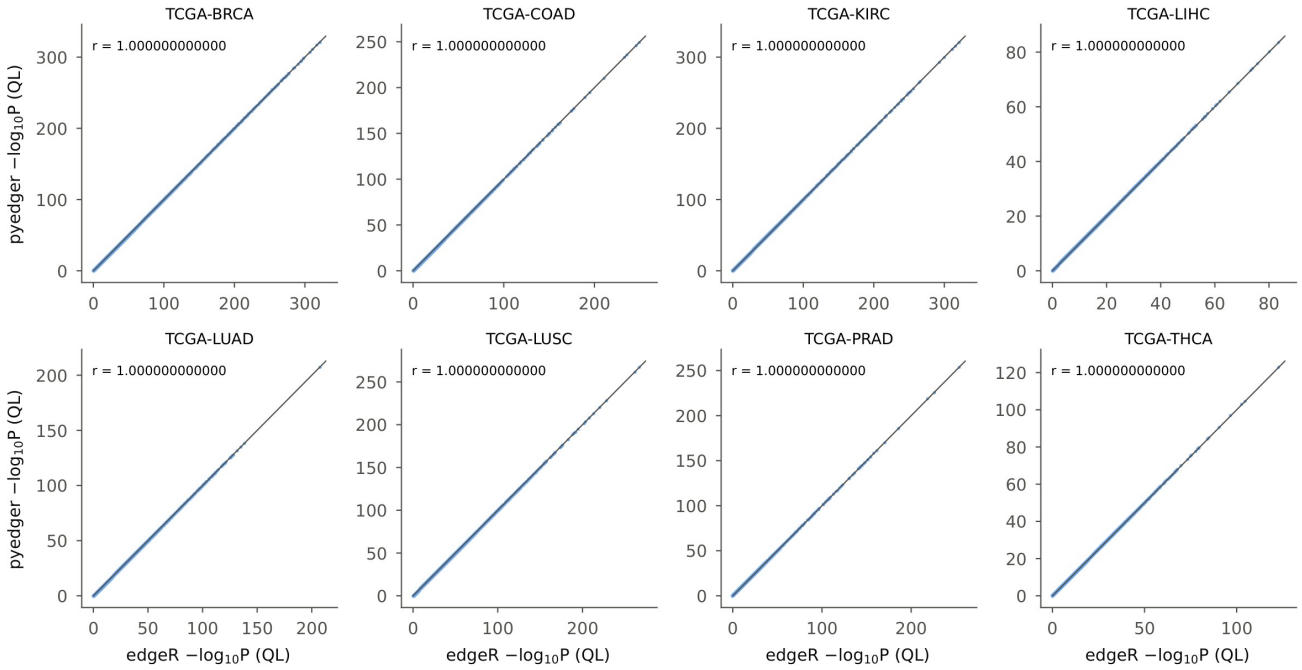


Figure S1. QL F-test P-values of pyedger against edgeR.

$-\log_{10} P$ of pyedger 0.2.3 (y) against edgeR 4.10.5 (x) for every retained gene of the eight TCGA cohorts; the line is the identity and r the Pearson correlation.